

ILAFD: Accuracy-Configurable Floating-Point Divider Using an Approximate Reciprocal and an Iterative Logarithmic Multiplier

James Oelund
University of Washington
Bothell, Washington, USA
jioelund@uw.edu

Sunwoong Kim
University of Washington
Bothell, Washington, USA
sunwoong@uw.edu

ABSTRACT

Approximate computing provides benefits in the logic area, latency, and/or power without significantly affecting the outputs of some applications. Various approximate computing techniques have been applied for floating-point (FP) dividers that are expensive compared to other FP operators. Since the accuracy requirement varies depending on the application, this paper proposes an accuracy-configurable approximate FP divider design. This design computes the reciprocal of a divisor in a hardware-friendly way. To correct approximation errors, multiple error biases are calculated based on error analysis and stored in a lookup table. The calculated reciprocal is multiplied by a dividend. For accuracy-configurability, an iterative logarithmic FP multiplier is adopted. Compared to previous approximate designs, our proposed design achieves a significantly higher level of accuracy by increasing the number of iterations, at the cost of a slight increase in hardware resources. Compared to the state-of-art Xilinx LogiCORE IP design for accurate FP division, the proposed approximate design reduces the number of lookup table blocks by 53% and the number of flip-flops by 90% in a field-programmable gate array. In JPEG compression, the conventional exact FP multiplier/divider and the proposed design with the number of iterations of 4 show almost the same peak signal-to-noise ratio values.

CCS CONCEPTS

• **Hardware** → **Arithmetic and datapath circuits; Hardware accelerators;** • **Theory of computation** → Approximation algorithms analysis.

KEYWORDS

approximate computing; field-programmable gate arrays; floating-point; iterative architecture; logarithmic divider

ACM Reference Format:

James Oelund and Sunwoong Kim. 2023. ILAFD: Accuracy-Configurable Floating-Point Divider Using an Approximate Reciprocal and an Iterative Logarithmic Multiplier. In *Proceedings of the Great Lakes Symposium on VLSI 2023 (GLSVLSI '23)*, June 5–7, 2023, Knoxville, TN, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3583781.3590262>

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
GLSVLSI '23, June 5–7, 2023, Knoxville, TN, USA
© 2023 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0125-2/23/06.
<https://doi.org/10.1145/3583781.3590262>

1 INTRODUCTION

Some real-world applications do not require exact results, and always-accurate arithmetic is wasteful [3, 22]. Allowing small computation errors often leads to advantages in logic area, latency, and/or power. This practical technique known as *approximate computing* is widely used. For example, small errors in image/video processing are undetectable by the human visual system [6, 7, 11, 15–17, 19, 23]. Machine learning inference is often tolerant of errors, and an output inference class is not significantly affected by small errors [7, 9, 13, 16].

A floating-point (FP) divider performs division between real numbers. It contains an integer divider called the mantissa divider, which occupies the majority of the FP divider logic area [16, 19]. Specifically, the IEEE 754 single-precision FP (SPFP) and double-precision FP (DPFP) standard formats require 23-bit and 52-bit unsigned integer dividers, respectively [4]. There have been several studies to optimize the large integer dividers or replace them with simpler operators, based on approximate computing techniques. One of the methods is Mitchell's division algorithm [12]. It approximately converts division into a combination of addition, subtraction, and shift operations on the logarithmic number system (LNS) domain. This hardware-friendly nature has made the algorithm popular in custom hardware designs.

An approximate divider trades off accuracy with resource efficiency, and thus several error correction (EC) methods have been presented to mitigate generated errors. In general, EC methods have low computational complexity to maintain the gain of approximate dividers. In particular, there are one or a few modes, and the values of relevant parameters, such as the number of truncation bits, are fixed at design time. For example, FaNZed applies a pre-obtained negative error bias to Mitchell's divider and adjusts the accuracy level by changing the number of truncation bits at design time [16]. However, required accuracy ranges are different depending on a specific application, and even within the same application, they may vary over time. Therefore, it is important to give dynamic configurability to an approximate divider at *runtime* [11].

Despite the widespread use of FP division in recent applications, approximate FP dividers have received less attention compared to approximate FP multipliers [6–8, 11, 15, 19]. An area-efficient approximate FP divider has advantages over a fixed-point divider due to its wide dynamic range.

The contributions of this paper are as follows:

- We extend the existing iterative logarithmic FP multiplier design [10] to an FP divider, which we refer to as the iterative logarithmic approximate FP divider (ILAFD). In particular, the approximate reciprocal of a divisor is calculated in a lightweight way and multiplied by a dividend.

- We propose a novel EC method based on approximation error analysis. Multiple error biases are calculated and stored in a lookup table, and a method to reduce the lookup table size is additionally proposed.
- We propose a hardware architecture of ILAFD for a field-programmable gate array (FPGA) and evaluate the design in terms of accuracy, logic area, and latency. In addition, the results of applying ILAFD to JPEG compression are presented.

The remainder of this paper is organized as follows. Section 2 introduces the background for this paper. The baseline of our area-efficient approximate reciprocal calculation method is presented in Section 3. Section 4 shows the proposed lookup table-based EC method to improve the accuracy of the baseline. The hardware architecture of ILAFD is outlined in Section 5. The evaluation results are shown in Section 6, and Section 7 concludes this paper.

2 BACKGROUND

2.1 IEEE Standard for Floating-Point Arithmetic

The IEEE 754 FP formats are a common standard for the representation of real numbers in binary [4]. In particular, SPFP and DFPF are among the most widely used. They consist of three primary components: a bit to represent the sign (a zero for positive and a one for negative values), a range of bits to represent the exponent, and a range of bits to represent the mantissa (fractional portion). Although not stored as a value, the formats define the mantissa as consisting of a leading bit 1, meaning the mantissa represents values ranging from one to two. The representation of a real number x in an FP format is then expressed as follows:

$$x = (-1)^s \times 2^{e-bias} \times (1.f), \quad (1)$$

where s represents the sign value, e represents the exponent value, and the fraction f lies in the range $0 \leq f < 1$. Because the bits comprising the exponent value do not include a sign bit, the exponent acts as an unsigned integer. The *bias* constant used in (1) allows for the expression of negative exponent values. The *bias* value is defined based on the number of bits used to represent the exponent, as half the range of possible exponent values.

Although other number formats have been devised which allow for variable bit ranges to represent the exponent and mantissa, the IEEE 754 FP formats all use a fixed number of bits for both values, which enables simple hardware. For example, SPFP uses 8 bits for the exponent and 23 bits for the mantissa.

2.2 Logarithmic Multiplication and Division

Moving multiplication and division to the LNS domain provides an opportunity to replace the expensive operations with cheaper addition, subtraction, and shift operations. In the LNS domain, a number x is represented as follows:

$$\log_2(x) = l + \log_2(1 + r), \quad (2)$$

where l is the integer portion of the logarithm and r is a real number satisfying $0 \leq r < 1$. Multiplication and division between x_1 and x_2 are then represented as follows:

$$x_1 \cdot x_2 = 2^{l_1+l_2} \cdot (1+r_1) \cdot (1+r_2), \quad (3)$$

$$\frac{x_1}{x_2} = 2^{l_1-l_2} \cdot \frac{1+r_1}{1+r_2}. \quad (4)$$

Mitchell's multiplication/division algorithm uses the approximation of a logarithm $\log_2(1+r) \approx r$ [12]. Specifically, multiplication in the LNS domain is approximated as follows:

$$\log_2(x_1 \cdot x_2) \approx l_1 + l_2 + \log_2(1 + r_1 + r_2). \quad (5)$$

Applying the antilogarithm to (5) provides the following approximate equation.

$$x_1 \cdot x_2 \approx \begin{cases} 2^{l_1+l_2} \cdot (1 + r_1 + r_2), & r_1 + r_2 < 1 \\ 2^{l_1+l_2+1} \cdot (r_1 + r_2), & r_1 + r_2 \geq 1. \end{cases} \quad (6)$$

Using the same approach, the approximate division equations in the LNS domain and the antilogarithm domain are derived as (7) and (8), respectively.

$$\log_2\left(\frac{x_1}{x_2}\right) \approx l_1 - l_2 + \log_2(1 + r_1 - r_2), \quad (7)$$

$$\frac{x_1}{x_2} \approx \begin{cases} 2^{l_1-l_2-1} \cdot (2 + r_1 - r_2), & r_1 - r_2 < 0 \\ 2^{l_1-l_2} \cdot (1 + r_1 - r_2), & r_1 - r_2 \geq 0. \end{cases} \quad (8)$$

Although Mitchell's algorithm offers benefits, such as small hardware, it results in a large error. To solve this problem, various EC methods have been proposed. Iteration-based methods correct errors by reusing the same hardware [1, 10, 14]. They produce small errors in systems where operands do not come in every cycle. In addition, they adjust latency and accuracy at runtime by changing the number of iterations. Babić *et al.* proposed an iterative method for logarithmic integer multiplication [1]. The error of Mitchell's multiplication algorithm during the first iteration is $2^{l_1+l_2} \cdot r_1 \cdot r_2$, which is the difference between (3) and (6). During the second iteration, $2^{l_1} \cdot r_1 (= x_1 - 2^{l_1})$ and $2^{l_2} \cdot r_2 (= x_2 - 2^{l_2})$ are used as operands, and the multiplier repeats the calculation in (6). Since the product of the second iteration is an approximation of the error of the first iteration, adding up the two products reduces the total error. If the product for the i -th iteration is denoted by $P_i(x_1 \cdot x_2)$, the final multiplication result with n iterations is calculated as follows:

$$x_1 \cdot x_2 \approx \sum_{i=1}^n P_i(x_1 \cdot x_2). \quad (9)$$

Recent works extend Babić *et al.*'s approach to the IEEE 754 SPFP format and the posit format [10, 14]. To this end, separate logic for components of those formats is generated, which is not required for integer multiplication. In Kim and Rutenbar's work, on which our ILAFD design is based, only a single operand is approximated to reduce hardware resources (e.g., leading one detector), and lightweight methods to mitigate the increase in error from this modification are proposed. For more details about these optimization techniques, refer to [10].

3 BASELINE FOR CALCULATING APPROXIMATE RECIPROCALLS

This section proposes a lightweight method to compute an approximate reciprocal of an FP number. Suppose that an FP number y has exponent e_1 and fraction f_1 and that $1/y$ has exponent e_2 and fraction f_2 . Based on that $y \cdot (1/y) = 1$ and that the FP number

of 1 has an exponent of $bias$ and a mantissa of 1.0, the following equations are derived after normalization.

$$e_1 + e_2 + 1 - bias = bias, \quad (10)$$

$$(1 + f_1) \cdot (1 + f_2)/2 = 1. \quad (11)$$

After transposing, (10) and (11) are converted into (12) and (13), respectively.

$$e_2 = 2 \cdot bias - 1 - e_1, \quad (12)$$

$$1 + f_2 = 2/(1 + f_1). \quad (13)$$

Since $2 \cdot bias - 1$ is a constant, a subtraction operation is only needed to compute e_2 . On the other hand, the mantissa computation of $1/y$ requires a division operation. To remove this division operation, a variant of the Taylor series is applied, and the right-hand side of (13) is approximated as follows:

$$2/(1 + f_1) \approx 2 \cdot (1 - f_1/2) = 2 - f_1. \quad (14)$$

In the binary number system, $2 - f_1$ is calculated in two steps: flipping bits of f_1 (with the hidden bit of 0) and adding 1. We omit the second step because it has little effect on the approximation error. In summary, the approximate fraction of $1/y$ is computed by simply negating the fraction of y .

4 VARIABLE ERROR CORRECTION

In this section, the EC method that works in tandem with the baseline is proposed. Using the baseline method alone is a fast and area-efficient solution but does not provide the accuracy needed to properly utilize the functionality of the iterative multiplier. Over the range of all possible values for the input mantissa, the mean error in the mantissa of the approximate reciprocal, denoted by E_{rcp} , is calculated using (15).

$$E_{rcp} = \frac{1}{2} \int_0^1 (2 - f_1) - \frac{2}{1 + f_1} df_1. \quad (15)$$

Compared to the expected value of mantissas of correct reciprocals, the error rate is 8.3%.

The error in the baseline always has a positive value. As presented in [16], shifting approximation errors by a constant EC value reduces the mean error. Specifically, if the calculated EC value from (15) is subtracted from approximation errors of the baseline, the error rate is reduced from 8.3% to 3.4%.

To achieve higher accuracy beyond this static EC method, a different amount for EC is selected based on the most significant h bits of the fraction, which serve as an index for a lookup table. For instance, if the h value is 4, this method results in a possible choice of sixteen different EC values from the table. To determine the EC values for the lookup table, the mean error for each of the possible indexed ranges, denoted by E'_{rcp} , is calculated as follows:

$$E'_{rcp} = 2^{h-1} \int_{\frac{i}{2^h}}^{\frac{i+1}{2^h}} (2 - f_1) - \frac{2}{1 + f_1} df_1, \quad (16)$$

where $0 \leq i < 2^h$ represents the value of the most significant h bits of the fraction.

Table 1 shows the EC values used in the lookup table, based on (16), when the h value is 4. In the hardware implementation,

Table 1: Lookup Table Values for Our Variable EC ($h = 4$)

i	EC-val[i]	i	EC-val[i]
0	0.0144042968750	8	0.0811767578125
1	0.0385742187500	9	0.0755615234375
2	0.0567626953125	10	0.0679931640625
3	0.0699462890625	11	0.0587158203125
4	0.0787353515625	12	0.0478515625000
5	0.0837402343750	13	0.0356445312500
6	0.0855712890625	14	0.0222167968750
7	0.0845947265625	15	0.0075683593750*

*Approximated to 0 to eliminate underflow cases.

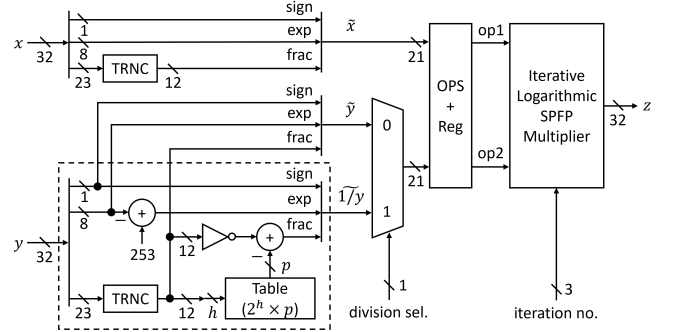


Figure 1: Overall architecture of ILAFD for the SPFP format.

these EC values are coded as binary numbers. Exceptionally, for mantissa values in the index range of 15, EC is omitted. This is done to avoid the need for underflow detection, as the mean error of the reciprocal approximation for fraction values starting with 1111₂ is relatively low. Also, all EC values in the table are left shifted by 1 due to the normalization of the baseline. Using this EC approach in combination with the baseline allows for a closer approximation to the true reciprocal curve over the range of all possible input mantissa values than would be possible with a single EC value.

As an example of how the proposed EC method functions, consider an SPFP number of 0.574711 for the divisor, with a mantissa of (1).001001100100₂ (last 11 bits truncated). The true reciprocal of this SPFP number is 1.740004976 (rounded). The baseline results in a mantissa of (1).110110011011₂, corresponding to a decimal value of 1.850341797 (rounded). Because the most significant four bits of the original fraction are 0010₂, the EC value from index 2 is used from the table. After the EC value of 0.113525390625 (=EC-val[2] ≪ 1) is subtracted from the baseline calculation, the approximate reciprocal becomes 1.736816406375, which means that the error decreases from 6.34% to 0.18%.

5 OVERALL HARDWARE ARCHITECTURE

Figure 1 shows the overall architecture of ILAFD for the IEEE 754 SPFP format, which performs multiplication as well. Although we explain the architecture with the SPFP format, the proposed approach is applicable to other FP formats. The logic blocks for approximate reciprocal calculation are contained within the dotted rectangle. When a divisor enters, an unsigned integer subtractor computes the exponent of the reciprocal, based on (12). For the

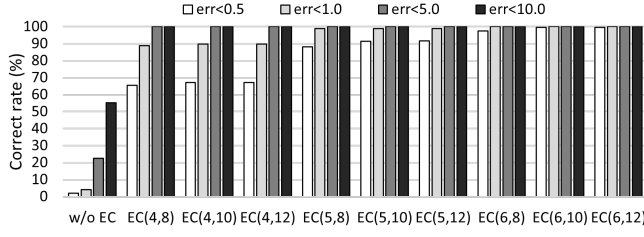


Figure 2: Correct rates of the proposed approximate reciprocal calculation method itself without EC (baseline) and with EC using parameters (h, p) .

fraction, the least significant t bits are truncated (TRNC) to reduce hardware resources. The value of t is determined at design time. It is set to 11 based on [10], which has little effect on the accuracy level. As described in Section 3, the fraction of the reciprocal of a divisor is first computed using negation. The most significant h bits of the fraction of the divisor are entered into the lookup table as an index. To reduce the lookup table size, truncation is applied to EC values, and each EC value is represented using p bits. Consequently, the size of the lookup table is $2^h \times p$ bits. In our design, the lookup table is implemented using logic. The EC value corresponding to the index is read and then subtracted from the negated fraction.

Depending on the type of operation (i.e., multiplication or division), either the original operand with truncation ($\tilde{y}$ in this figure) or the approximate reciprocal ($\frac{1}{\tilde{y}}$) is selected. The iterative logarithmic SPFP multiplier design then multiplies it by another truncated operand ($\tilde{x}$) according to the number of iterations entered. Basically, the multiplier design is the same as [10], but there is a modification: the operand selector (OPS) in the multiplier design is moved ahead of operand registers to shorten the critical path delay.

6 EVALUATION

This section evaluates the accuracy of ILAFD. In addition, the FPGA implementation results are presented. Lastly, we demonstrate the effectiveness of ILAFD in JPEG compression.

6.1 Accuracy

To measure the accuracy, the *correct rate* used in [1, 10] is calculated. Specifically, an error in percentage, denoted by *err*, is calculated by dividing the absolute difference between the exact and approximate results by the exact result. After executing 200 million divisions with random SPFP numbers, it is checked which range the calculated error value falls into. We use $\text{err} < 0.5\%$, $< 1.0\%$, $< 5.0\%$, and $< 10.0\%$ as ranges. Percent signs for the ranges are omitted from the rest of this paper. Finally, the correct rate in percentage is obtained by dividing the count value for each error range by 200 million.

Figure 2 shows the correct rates of the proposed approximate reciprocal calculation method itself without EC (baseline) and with EC, depending on the error range. As explained in Sections 4 and 5, the proposed EC method uses two parameters, h and p , and the notation $\text{EC}(h, p)$ is used in this section. To save space, only nine combinations of h and p values are presented: the h values of 4, 5, and 6 and the p values of 8, 10, and 12. As shown in the first set of bar graphs, the baseline produces large approximation errors.

Table 2: Accuracy Comparison of Approximate SPFP Dividers

	n	Correct rate (%)			
		err<0.5	err<1.0	err<5.0	err<10.0
Mitchell [12] [†]	-	12.02	20.65	64.50	93.82
FaNZeD [16] [†]	-	8.91	17.90	87.02	100.00
FPAD-L4A3 [7] [†]	-	14.11	26.82	96.27	100.00
LPCAD-K3 [19] [†]	-	53.87	74.25	99.96	100.00
ILAFD-EC(4,10)	2	23.53	43.03	95.42	100.00
	3	54.30	82.43	100.00	100.00
	4	66.50	89.68	100.00	100.00
	5	67.10	89.75	100.00	100.00
	5	67.10	89.75	100.00	100.00
ILAFD-EC(5,10)	2	28.37	48.59	95.75	100.00
	3	73.42	93.51	100.00	100.00
	4	90.33	98.97	100.00	100.00
	5	91.30	98.98	100.00	100.00
	5	91.30	98.98	100.00	100.00
ILAFD-EC(6,10)	2	31.36	50.15	95.79	100.00
	3	81.40	95.00	100.00	100.00
	4	99.14	100.00	100.00	100.00
	5	99.50	100.00	100.00	100.00
	5	99.50	100.00	100.00	100.00

[†] 11-bit truncation is applied.

Specifically, the correct rate is 2% in the range $\text{err} < 0.5$. Adding the proposed EC method to the baseline significantly increases correct rates. In particular, increasing the h value, which provides more EC values, has a huge impact. When the h value is 4, the correct rates in the range $\text{err} < 0.5$ are 67%. When this value increases to 5, the correct rates in the same error range becomes 90%. Finally, setting the h value to 6 increases the correct rates in the error range to 99%.

Reducing the p value from 12 to 10 does not significantly affect the correct rate of a reciprocal. Specifically, the decrease is less than 0.1%. However, as the p value decreases from 10 to 8, the decrease in the correct rate becomes 2%, which is a relatively large increase, and thus 10 is chosen as the p value in the hardware implementation.

Table 2 shows the correct rate results of ILAFD. For comparison, Mitchell's divider [12], of which the formula is shown in (8), FaNZeD [16], FPAD [7], and LPCAD [19] are used. FaNZeD is almost the same as Mitchell's divider but subtracts an analytically deduced error bias from the result of Mitchell's divider. Specifically, $2^{h_1-h_2} \cdot \epsilon$ is subtracted from (8), and the value of ϵ is $2^{-5} + 2^{-7}$. FPAD approximates the reciprocal of a divisor as the sum of $A+1$ powers of two. In particular, like ILAFD, 2^L cases are created using the most significant L bits of the fraction of a divisor, and different powers of two are used for each case. In this section, the L and A values are set to 4 and 3, respectively. LPCAD is a logarithmic FP divider. It approximates $\log_2(\frac{1+r_1}{1+r_2})$ to a piecewise linear function. In addition, depending on the value of the most significant K bits of the fraction of a divisor, $\frac{1}{1+r_2}$ is approximated by a different constant. The K value used in this section is 3. There is a modification to these previous works: 11-bit truncation that is used in ILAFD is applied for a fair comparison. ILAFD uses the (h, p) values of (4, 10), (5, 10), and (6, 10) for EC. Unlike the previous non-iterative SPFP dividers, it uses iterations in the multiplier, and the number of iterations n is shown in the second column of Table 2.

Mitchell's divider that does not use EC shows the lowest correct rates in divisions. FaNZeD improves the correct rates in the ranges

Table 3: FPGA Implementation Results

	LogiCORE [20]	FaNZeD [‡] [16]	FPAD-L4A3 [‡] [7]	LPCAD-K3 [‡] [19]	SAADI-EC [11]	ILAFD EC(6, 10)
Format	SPFP	SPFP	SPFP	SPFP	8-bit fixed	SPFP
Architecture	Pipelined	Single-cycle	Single-cycle	Single-cycle	Iterative	Iterative
f_{max} (MHz)	>770	750	500	570	340	650
LUT	761	115	279	111	323	357
FF	1,387	22	33	26	38	144
Latency	28 cycles	1 cycle	1 cycle	1 cycle	$n+2$ cycles	$n+3$ cycles

[‡]11-bit truncation is applied.

$err < 5.0$ and $err < 10.0$ to 87% and 100%, respectively. However, in the ranges $err < 0.5$ and $err < 1.0$, it performs worse on correct rates because the error bias value greater than an approximation error value is often subtracted. FPAD shows a correct rate of 96% in the range $err < 5.0$ due to multiple approximation options. However, each approximation option is not elaborated, so the correct rates do not increase significantly in the ranges $err < 0.5$ and $err < 1.0$ compared to Mitchell's divider and FaNZeD. LPCAD, which performs piecewise approximation with multiple approximation options, significantly improves the correct rates of the aforementioned previous dividers in the ranges $err < 0.5$ and $err < 1.0$.

For ILAFD, the approximation error of a reciprocal decreases as the value of h increases, as shown in Figure 2. It results in the increase of correct rates in divisions. Note that h values greater than 6 do not significantly change correct rates. The correct rates in divisions are improved as the n value increases. When the value of n is greater than 2, ILAFD always shows higher correct rates than LPCAD. The correct rates of ILAFD converge when the n value reaches 5, and they are almost the same as ideal values that are the correct rates of our approximate reciprocals.

6.2 Hardware Implementation

Table 3 shows the FPGA implementation results of real number divider designs. All designs were described using the Verilog hardware description language. Xilinx Vivado Design Suite (2020.1) was used for synthesis and place-and-route stages targeting the same Xilinx Virtex UltraScale+ FPGA (xcvu9p) device that has 1M lookup table blocks (LUTs) and 2M flip-flops (FFs).

As a previous design for comparison, the state-of-art Xilinx LogiCORE IP of an SPFP divider was used [20]. This design performs exact division except for rounding errors and is based on a pipelined architecture. Other previous SPFP divider designs are FaNZeD, FPAD, and LPCAD, which are introduced in the previous subsection. They are based on single-cycle architectures. Note that Mitchell's divider is almost identical to FaNZeD, so its FPGA implementation results are not shown in Table 3. Another previous divider design is the work of Melchert *et al.*, called SAADI-EC [11]. This design uses iterations to compute the approximate reciprocal of a divisor and multiplies it by the dividend. A division result is scaled to correct approximation errors. SAADI-EC is identical to the proposed ILAFD design in that it uses iterations at runtime to trade off accuracy with latency. However, it performs fixed-point division, and iterations are exploited for not multiplication but reciprocal calculation.

The ILAFD design uses 6 and 10 for the h and p values, respectively. It uses shifters for multiplication and division, so it does

not utilize a digital signal processing (DSP) slice with a high-speed multiplier block. For a fair comparison, the previous designs were implemented without using DSP slices. Note that the number of DSP slices is usually more limited than the number of LUTs, so operators using only LUTs and FFs benefit from scalability in FPGAs.

The LogiCORE IP design uses the largest number of LUTs, especially for the mantissa division, in Table 3. In addition, it requires the largest number of FFs and the longest latency due to a large number of pipeline stages. These are wasteful in applications that do not require high throughput and accurate computation. The three previous approximate SPFP divider designs based on single-cycle architectures utilize relatively small hardware resources, specifically consuming on average 47% of the LUTs and 19% of the FFs of the ILAFD design. However, as shown in Table 2, these designs show lower accuracy than ILAFD when the n value exceeds 2, and are unable to adjust accuracy dynamically.

The number of LUTs and the latency of SAADI-EC are similar to those of the proposed design. However, SAADI-EC uses relatively many LUTs, considering that the bit-width of operands is 8 and fixed-point division is performed. This is because SAADI-EC uses iterations for approximate reciprocal calculations and still needs an expensive real number multiplier. Typically, the number of hardware resources of a real number multiplier is greater than that of a reciprocal calculator in FPGAs [21].

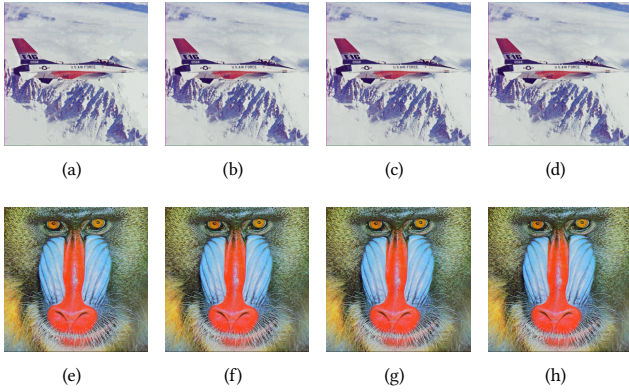
6.3 Case Study: JPEG Compression

JPEG compression is utilized to assess ILAFD in an application where a certain degree of error tolerance is acceptable. Specifically, the conventional IEEE 754 SPFP multiplier and divider for discrete cosine transform and scaling in JPEG are replaced by ILAFD. For implementation, the TooJpeg open-source library is used [2]. Six popular 512×512-sized images with three color components are used as inputs to the JPEG: airplane, house, lake, lenna, mandrill, and pepper [5, 18]. The value of the quality level, which is one of the JPEG parameters, is set to 90 (%). This value causes data loss but guarantees a very high-quality image.

To measure the similarity between the original uncompressed images and the compressed images, the peak signal-to-noise ratio (PSNR) values were calculated, which are shown in Table 4. For comparison, the PSNR results when the conventional SPFP multiplier and divider are used are presented in the second column of the table. When ILAFD is used, the PSNR values increase as the number of iterations n increases. In particular, when the n value is 4, the PSNR values are almost the same as when using the conventional SPFP multiplier and divider. Figure 3 shows the resulting images

Table 4: PSNR (dB) Results of the JPEG Compressed Images

Image	IEEE 754 SPFP Conv.	ILAFD-EC(6,10)			
		$n = 2$	$n = 3$	$n = 4$	$n = 5$
airplane	38.54	35.41	38.37	38.53	38.54
house	36.47	34.56	36.39	36.47	36.47
lake	32.85	31.64	32.79	32.84	32.85
lenna	36.25	34.61	36.18	36.24	36.25
mandrill	31.33	30.58	31.30	31.33	31.33
pepper	34.40	33.01	34.33	34.39	34.40
Average	34.97	33.30	34.89	34.97	34.97

**Figure 3: Resulting images from JPEG using ILAFD. (a) airplane with n of 2 (b) airplane with n of 3 (c) airplane with n of 4 (d) airplane with n of 5 (e) mandrill with n of 2 (f) mandrill with n of 3 (g) mandrill with n of 4 (h) mandrill with n of 5.**

for airplane and mandrill, where the JPEG shows the highest and lowest PSNR values, respectively.

7 CONCLUSION

In this paper, we presented an accuracy-configurable FP divider design for an FPGA. It computes an approximate reciprocal of a divisor in a lightweight way and corrects errors using multiple EC values stored in a lookup table. The computed reciprocal is multiplied by a dividend using the existing iterative logarithmic FP multiplier. Since the number of iterations is configurable at runtime, trading off accuracy with latency is available depending on the applications. The ILAFD design outperforms the previous approximate FP divider designs based on single-cycle architectures in terms of accuracy when the n value exceeds 2, at the cost of a slight increase in hardware resources. Compared to the Xilinx LogiCORE IP design of an SPFP divider, which performs exact division, the ILAFD design reduces the number of LUTs, number of FFs, and latency by 53%, 90%, and 71%, respectively. To the best of the authors' knowledge, this is the first work that uses an iterative logarithmic FP multiplier design for approximate FP divisions.

ACKNOWLEDGMENTS

We thank Dr. Rob Rutenbar, Dr. Jung Hee Cheon, Wonhee Cho, and Keewoo Lee. This work was supported by Institute of Information & Communications Technology Planning & Evaluation (IITP)

grant funded by the Korea government (MSIT) (No. 2020-0-00840, Development and Library Implementation of Fully Homomorphic Machine Learning Algorithms supporting Neural Network Learning over Encrypted Data).

REFERENCES

- [1] Zdenka Babić, Aleksej Avramović, and Patricio Buli. 2011. An iterative logarithmic multiplier. *Microprocessors and Microsystems* 35, 1 (2011), 23–33.
- [2] Stephan Brumme. 2019. *A JPEG encoder in a single C++ file*. <https://github.com/stbrumme/toojpeg>
- [3] Jie Han and Michael Orshansky. 2013. Approximate computing: An emerging paradigm for energy-efficient design. In *2013 18th IEEE European Test Symposium (ETS)*. IEEE, 1–6.
- [4] IEEE. 2019. IEEE Standard for Floating-Point Arithmetic. *IEEE Std. 754-2019 (Revision of IEEE 754-2008)* (2019), 1–84.
- [5] ImageProcessingPlace. 2008. *Image Databases*. https://www.imageprocessingplace.com/root_files_V3/image_databases.htm
- [6] Mohsen Imani, Ricardo Garcia, Andrew Huang, and Tajana Rosing. 2019. Cade: Configurable approximate divider for energy efficiency. In *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 586–589.
- [7] Chandan Kumar Jha, Kailash Prasad, Vibhor Kumar Srivastava, and Joyce Meki. 2020. FPAD: a multistage approximation methodology for designing floating point approximate dividers. In *2020 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–5.
- [8] Honglan Jiang, Cong Liu, Leibo Liu, Fabrizio Lombardi, and Jie Han. 2017. A review, classification, and comparative evaluation of approximate arithmetic circuits. *ACM Journal on Emerging Technologies in Computing Systems (JETC)* 13, 4 (2017), 1–34.
- [9] Sunwoong Kim and Rob Rutenbar. 2018. Accelerator Design with Effective Resource Utilization for Binary Convolutional Neural Networks on an FPGA. In *2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 218–218.
- [10] Sunwoong Kim and Rob A Rutenbar. 2019. An area-efficient iterative single-precision floating-point multiplier architecture for FPGA. In *Proceedings of the 2019 on Great Lakes Symposium on VLSI*. 87–92.
- [11] Jackson Melchert, Setareh Behrooz, Jingjie Li, and Younghyun Kim. 2019. SAADI-EC: A quality-configurable approximate divider for energy efficiency. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 27, 11 (2019), 2680–2692.
- [12] John N Mitchell. 1962. Computer multiplication and division using binary logarithms. *IRE Transactions on Electronic Computers* 4 (1962), 512–517.
- [13] Raul Murillo, Alberto A Del Barrio, Guillermo Botella, Min Soo Kim, HyunJin Kim, and Nader Bagherzadeh. 2021. PLAM: A posit logarithm-approximate multiplier. *IEEE Transactions on Emerging Topics in Computing* 10, 4 (2021), 2079–2085.
- [14] Cameron James Norris and Sunwoong Kim. 2021. An approximate and iterative posit multiplier architecture for FPGAs. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–5.
- [15] Omkar G Ratnaparkhi and Madhav Rao. 2022. LEAD: Logarithmic Exponent Approximate Divider For Image Quantization Application. In *Proceedings of the Great Lakes Symposium on VLSI 2022*. 437–442.
- [16] Hassaan Saadat, Haris Javaid, and Sri Parameswaran. 2019. Approximate integer and floating-point dividers with near-zero error bias. In *Proceedings of the 56th Annual Design Automation Conference 2019*. 1–6.
- [17] Shaghayegh Vahdat, Mehdi Kamal, Ali Afzali-Kusha, Massoud Pedram, and Zainalabedin Navabi. 2017. TruncApp: A truncation-based approximate divider for energy efficient DSP applications. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2017. IEEE, 1635–1638.
- [18] Allan G Weber. 2006. The USC-SIPI image database: Version 5. <http://sipi.usc.edu/database/> (2006).
- [19] Yong Wu, Honglan Jiang, Zining Ma, Pengfei Gou, Yong Lu, Jie Han, Shouyi Yin, Shaojun Wei, and Leibo Liu. 2022. An Energy-Efficient Approximate Divider Based on Logarithmic Conversion and Piecewise Constant Approximation. *IEEE Transactions on Circuits and Systems I: Regular Papers* 69, 7 (2022), 2655–2668.
- [20] Xilinx. 2020. *LogiCORE IP Floating-Point Operator v7.1 Product Guide (PG060)*. https://www.xilinx.com/support/documentation/ip_documentation/floating_point/v7_1/pg060-floating-point.pdf
- [21] Xilinx. 2020. *Performance and Resource Utilization for Floating-point v7.1*. https://www.xilinx.com/htmldocs/ip_docs/pru_files/floating-point.html
- [22] Qiang Xu, Todd Mytkowicz, and Nam Sung Kim. 2015. Approximate computing: A survey. *IEEE Design & Test* 33, 1 (2015), 8–22.
- [23] Reza Zendegani, Mehdi Kamal, Arash Fayyazi, Ali Afzali-Kusha, Saeed Safari, and Massoud Pedram. 2016. SEERAD: A high speed yet energy-efficient rounding-based approximate divider. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1481–1484.